

Report Markdown

The script to download OpenAlex data

The first script is used to obtain the author names for the purpose of searching them in the OpenAlex database. This is done as there is no specific identifier in the PURE database linking the researchers to OpenAlex. This will present a problem with duplicate names that will get resolved by matching the researchers' output with the output in PURE.

```
external_names <- nodes[grepl("^EXT_", nodes$id),]  
  
author_name <- external_names$label  
  
author_name <- author_name[author_name != "NA NA"]  
  
saveRDS(author_name, file = "author_name.rds")
```

The main script was a loop that run the logic over 1000 researcher names at a time. This was done in order to limit the possibility of a problem occurring and losing the pulled API data.

The script requires the papers (which are all PURE papers available) and the previously created author_name file.

Then, the url for the API is specified and the vectors used to store downloaded data are initialized.

```
library(httr)  
library(jsonlite)  
  
papers <- readRDS("papers.rds")  
author_name <- readRDS("author_name.rds")  
  
author_name <- author_name[1:1000]
```

```

search_name <- URLencode(author_name, reserved = TRUE)
search_url <- paste0("https://api.openalex.org/authors?search=", search_name)

countries <- numeric(length(author_name))
uni <- numeric(length(author_name))

for (i in 1:length(search_url)) {
  response <- GET(search_url[i])
  search_results <-
    fromJSON(content(response, as = "text", encoding = "UTF-8"))
  res <- search_results$results

  results_for_name <- list()

  if (is.data.frame(res) && nrow(res) != 0) {
    for (j in 1:nrow(res)) {
      results_for_name[[length(results_for_name)+1]] <-
        fromJSON(content(GET(res$works_api_url[j]),
                           as = "text", encoding = "UTF-8"))$results
    }
  } else {
    results_for_name[[length(results_for_name)+1]] <- NA
  }

  for (h in 1:length(results_for_name)) {
    if(length(res) != 0) {
      if(length(which(results_for_name[[h]]$title
                      %in% papers$title.value)) != 0) {
        if(length(res$last_known_institutions[[h]]$display_name) != 0) {
          uni[i] <- (res$last_known_institutions[[h]]$display_name[1])
          countries[i] <- (res$last_known_institutions[[h]]$country_code[1])
        }
        else{
          uni[i] <- 0
          countries[i] <- 0
        }
      }
      break
    }
  }
}

rm(results_for_name)

```

```

}

downloaded_institutions <-
  data.frame(name = author_name, countries = countries, institutions = uni)

saveRDS(downloaded_institutions, file = "downloaded_institutions_1_100_test.rds")

```

To break down the logic of the main loop, the specific sub-loops will be explained

```

response <- GET(search_url[i])
search_results <- fromJSON(content(response, as = "text", encoding = "UTF-8"))
res <- search_results$results

```

This chunk is responsible for the download of all profiles on OpenAlex that match the provided name. The result for this is a dataframe with the nrow equal to the number of profiles retrieved.

```

results_for_name <- list()
if (is.data.frame(res) && nrow(res) != 0) {
  for (j in 1:nrow(res)) {
    results_for_name[[length(results_for_name)+1]] <-
      fromJSON(content(GET(res$works_api_url[j]),
                          as = "text", encoding = "UTF-8"))$results
  }
} else {
  results_for_name[[length(results_for_name)+1]] <- NA
}

```

The next chunk is in charge of retrieving all articles available on OpenAlex written by all profiles that were retrieved in the previous step. The output is in the form of a list.

```

for (h in 1:length(results_for_name)) {
  if(length(res) != 0) {
    if(length(which(results_for_name[[h]]$title
                     %in% papers$title.value)) != 0) {
      if(length(res$last_known_institutions[[h]]$display_name) != 0) {
        uni[i] <- (res$last_known_institutions[[h]]$display_name[1])
        countries[i] <- (res$last_known_institutions[[h]]$country_code[1])
      }
    } else{

```

```

        uni[i] <- 0
        countries[i] <- 0
      }
      break
    }
  }
}

```

The final loop is responsible for actually finding the institution of the researcher. It first runs through all of the outputs for each profile. It searches for the first profile index that contains an output that is available in the PURE data set. This is a way to ensure that the researcher is question is the specific researcher contained in PURE and not just a person with the same name. Then, after this has been identified, it retrieves the information about the institution and the country of the chosen profile and that concludes the entire step of the loop. The same process was then repeated for all available names.

After data, the vectors are merged into a dataframe and then saved in a .RDS file for further use.

Main app

Libraries and functions

```

library(shiny)
library(visNetwork)
library(leaflet)
library(this.path)
library(dplyr)

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

```
filter, lag
```

The following objects are masked from 'package:base':

```
intersect, setdiff, setequal, union
```

```

library(scales)
library(tidygeocoder)
library(countrycode)
library(fuzzyjoin)

setwd(this.dir())
`%nin%` <- Negate(`%in%`)

clean_name <- function(x) {
  x <- tolower(x)
  x <- gsub("[^a-z0-9\\s-]", "", x)
  x <- gsub("\\s+", " ", x)
  x <- trimws(x)
  return(x)
}

```

The file begins by calling all the necessary libraries and defining two functions. `%nin%` is simply a negation of the `%in%` function present in base R. This is only used once in the file, but was defined for ease of reading. `clean_name()` is a function used for the cleaning of university names. It is used in the fuzzy joining of the OpenAlex universities to the ones from QS Ranking.

Settings for leaflet

```

# -----
# 0) Settings
# -----
cache_file <- "inst_coords_cache.rds"
vienna_name <- "Vienna University of Economics and Business"

```

Loading and preparing the data

This section is focused on loading the data and merging it in order to pass it into the server and app logic. This mostly focuses on the edges and nodes, created in a section above, and on the institution information downloaded from OpenAlex and QS.

```

# -----
# 1) Load & prepare data
# -----

```

```

nodes      <- read.csv2("author_nodes_new.csv", stringsAsFactors = FALSE)
edges      <- read.csv2("author_edges_new.csv",   stringsAsFactors = FALSE)
institutions <- readRDS("downloaded_institutions_combined.rds")

qsranking <- read.csv("qsranking.csv")
qsranking <- subset(qsranking, select = c(index, rank_2025, rank_2024, institution_name, location))
qsranking$location <- countrycode(qsranking$location, origin = "country.name.en", destination = "country.iso2c")
colnames(qsranking) <- c('index', 'rank_2025', 'rank_2024', 'institutions', 'location')
colnames(edges) <- c("from", "to", "freq")

```

The data is loaded and preprocessed. For the *qsranking* file only a few columns are chosen. The country names are encoded into iso2c (eg. Austria = AT) to make the use with the OpenAlex more comfortable. The columns are also renamed to make the structure more standardized.

```

nodes$author_id <- as.character(nodes$author_id)
edges$from <- as.character(edges$from)
edges$to <- as.character(edges$to)

```

The *as.character()* function is used in order to ensure these variables are strings, this is essential to the functioning of the app.

```

# attach external EXT_ IDs to institution records
external_ids <- nodes[grepl("^EXT_", nodes$id) & nodes$label!="NA NA", ]$id
institutions$id <- external_ids

```

IDs are reattached to the corresponding names in the *institutions* dataframe.

```

# build internal "WU" entries
internal_names <- nodes[grepl("^A_", nodes$id), ]$label
internal_ids <- nodes[grepl("^A_", nodes$id), ]$author_id
internal_institutions <- data.frame(
  label      = internal_names,
  countries  = "AT",
  institutions = "Vienna University of Economics and Business",
  id         = internal_ids,
  stringsAsFactors = FALSE
)

```

A dataframe is created for the internal authors. This is necessary as an initial step to attach the data on institutions to the nodes dataframe.

```
# dedupe legacy institutions by (id,label)
institutions <- institutions %>%
  rename(label = name) %>%
  distinct(id, label, .keep_all = TRUE)

final_institutions <- bind_rows(internal_institutions, institutions) %>%
  distinct(id, label, .keep_all = TRUE)
```

The data is merged and filtered to only keep distinct IDs. This is not only done in order to avoid duplicates, but is necessary make the app run, as visNetwork does not accept any duplicate IDs.

```
# fix Unknown fields
final_institutions$countries [final_institutions$countries == 0] <- "Unknown"
final_institutions$institutions [final_institutions$institutions == 0] <- "Unknown"
colnames(final_institutions) <- c("label","countries","institutions","id")

# merge nodes → authors + institution info
nodes <- nodes %>%
  filter(label!="NA NA") %>%
  inner_join(final_institutions,
    by = c("id","label"),
    relationship = "many-to-many") %>%
  distinct(author_id, .keep_all = TRUE)
```

The zeros in the institutions dataframe are replaced with “Unknown”. After that, the dataframe is merged with the nodes using an inner join along the ID and name.

```
nodes$continent <- countrycode(sourcevar = nodes[, "countries"],
  origin = "iso2c",
  destination = "continent")
```

Warning: Some values were not matched unambiguously: UNKNOWN

```
nodes$continent[which(is.na(nodes$continent))] <- "Unknown"
```

This chunk uses the *countrycode* library to add continents to the nodes based on the country that was added with the merge.

```

#qs implementation
nodes_qsr ranking <- subset(nodes, select = c(id, institutions, countries))

nodes_qsr ranking$institutions <- clean_name(nodes_qsr ranking$institutions)
qsr ranking$institutions <- clean_name(qsr ranking$institutions)

result <- stringdist_left_join(nodes_qsr ranking, qsr ranking, by = "institutions", method = "jw")
result <- result %>% group_by(id) %>% filter(countries == location | countries == "Unknown")
rm(nodes_qsr ranking)

#cleaning qsr ranking results
result$rank_2025 <- gsub('[+]', '', result$rank_2025)
result$rank_2024 <- gsub('[+]', '', result$rank_2024)
result$rank_2025_num <- as.numeric(sapply(strsplit(result$rank_2025, c('-')), "[[", 1))
result$rank_2024_num <- as.numeric(sapply(result$rank_2024, function(x) {
  strsplit(x, '-')[[1]][1]
})))

```

This chunk shows how we use fuzzy join to match the names of the institutions from OpenAlex and QS. The basic idea is that the names of universities in both cases are cleaned and then a *jw* algorithm is run in order to check the distance between the strings. We only choose institutions if their countries match or are unknown and if the distance is less or equal to 0.2.

Lastly, we convert the rankings from strings to numeric.

```

#merging the cleaned results with nodes
result$rank <- (result$rank_2025_num + result$rank_2024_num)/2
result <- result[,c(1,12)]
nodes <- left_join(nodes, result, by = 'id')

```

The rank is calculated as the average of the ranks from 2025 and 2024. Then, the ranks are added to the nodes using a left join.

```

# collapse duplicate edges
edges <- edges %>%
  rowwise() %>%
  mutate(pair = paste(sort(c(from,to)), collapse=":")) %>%
  ungroup() %>%
  group_by(pair) %>%
  summarise(
    from = strsplit(pair,":")[[1]][1],

```



```

    to    = strsplit(pair,":")[[1]][2],
    freq = sum(freq),
    .groups = "drop"
)

```

The edges are collapsed if they show connections between the same IDs and the frequencies are summed when this happens. This is not needed, but makes the graphs a lot more readable.

```

# build author tooltip
nodes$title <- paste0(
  "Name: ", nodes$label,
  "<br>Publications: ", nodes$pubs,
  "<br>Country: ", nodes$countries,
  "<br>Continent: ", nodes$continent,
  "<br>Institution: ", nodes$institutions,
  "<br>Rank: ", nodes$rank
)

```

The tooltip for the author nodes is created from the variables of the *nodes* dataframe.

User Interface

```

# -----
# 3) UI
# -----
ui <- fluidPage(
  titlePanel(div(img(src="logo.png"), "Welcome to OPAL")),
  sidebarLayout(
    sidebarPanel(
      radioButtons("view_mode", "Top-level view:",
        choices = c("Author view", "Institution view"),
        inline = TRUE),
      uiOutput("edge_buttons"),
      uiOutput("dynamic_node_picker"),
      uiOutput("dynamic_net_switch"),
      uiOutput("inst_selector"),
      uiOutput("dynamic_slider"),
      uiOutput("rank_slider"),
      uiOutput("cont_checkbox")
    ),
  ),

```

```

mainPanel(
  conditionalPanel("input.view_mode=='Author view'",
    visNetworkOutput("filteredGraph",height="550px")),
  conditionalPanel("input.view_mode=='Institution view'",
    leafletOutput("inst_map",height="550px"))
)
)
)

```

The UI will not be discussed here as most of its features are store in dynamic UI elements defined in the server. The code just serves as a view as to how the app is structured.

Server

```

# -----
# 4) Server
# -----
server <- function(input, output, session) {}

```

All following code chunks should be in this server function, but for the sake of readability they were separated.

```

# -- edge drawing logic selection (connected vs. further) --
output$edge_buttons <- renderUI({
  req(input$view_mode=="Author view", input$selected_node)
  radioButtons("edge_mode","Edge-drawing logic:",
    choices=c("Simple connections","Further connections"),
    selected="Simple connections")
})

```

The first element is the output element that creates the input for the simple/further connections.

```

# -- dynamic author vs. institution selector --
output$dynamic_node_picker <- renderUI({
  if (input$view_mode=="Author view") {
    selectInput("selected_node","Select an author:",
      choices = nodes$label, selected = nodes$label[1])
  } else {

```

```

# only show institutions with coords
avail <- intersect(
  unique(nodes$institutions),
  inst_coord %>% filter(!is.na(latitude),!is.na(longitude)) %>% pull(address)
)
selectizeInput("selected_inst","Select an institution:",
  choices = sort(avail),
  selected = sort(avail)[1],
  options = list(server=TRUE))
}
})

```

This element is controlled by the input of the top view. If we choose the top view of authors, this will create the *selected_node* input, that is used to display the networks of different researchers. If the top view is institutions, this will create the *selected_inst* input, which is used to select institutions to show their connections on the map.

```

output$cont_checkbox <- renderUI({
  req(input$view_mode=="Author view", input$selected_node)
  checkboxGroupInput(
    "cont_filter", "Show continents:",
    choices = sort(unique(nodes$continent)),
    selected = sort(unique(nodes$continent))
  )
})

```

This creates the filter that allows to show nodes by continents. It requires the author view to be selected in order to show and function.

```

# -- institution rank filter (author view only) --
output$rank_slider <- renderUI({
  req(input$view_mode=="Author view", input$selected_node)
  sliderInput("rank_filter", "Max Rank:",
    min = min(na.omit(nodes$rank)), max = max(na.omit(nodes$rank)),
    value = max(na.omit(nodes$rank)))
})

```

This creates the filter that allows to show nodes by institution rank. It will completely ignore and not filter nodes without institution rank. It requires the author view to be selected in order to show and function.

```
# -- show-institutes filter (author view only) --
output$inst_selector <- renderUI({
  req(input$view_mode=="Author view", input$selected_node)
  ego_id <- nodes$author_id[nodes$label==input$selected_node]
  ce      <- subset(edges, from==ego_id|to==ego_id)
  ids     <- unique(c(ce$from,ce$to,ego_id))
  insts   <- nodes %>% filter(id%in%ids) %>% pull(institutions) %>% unique()
  checkboxGroupInput("inst_filter","Show institutions:",
                     choices=insts, selected=insts)
})
```

This creates the filter that allows for the display of nodes based on the institution. It detects and draws the list of institutions dynamically to only those that are available on the drawn network. It requires the author view to be selected in order to show and function.

```
# -- author-centric network switch --
output$dynamic_net_switch <- renderUI({
  req(input$view_mode=="Author view")
  radioButtons("net_mode","Author-centric network:",
              choices=c("People network","Institution network"),
              selected="People network")
})
```

This output creates an input element if the top view of authors is selected. It allows for the selection of a network showing the network of either authors or institutions connected to the selected nodes. The logic in how this is achieved is explained below.

```
# -- slider: pubs vs. collabs --
output$dynamic_slider <- renderUI({
  if (input$view_mode=="Author view") {
    sliderInput("pubs_filter","Max Publications:",
               min=min(nodes$pubs),
               max=max(nodes$pubs),
               value=max(nodes$pubs),
               step=1)
  } else {
    req(input$selected_inst)
    maxc <- inst_edges %>%
      filter(inst_from==input$selected_inst|inst_to==input$selected_inst) %>%
      pull(freq) %>% max(na.rm=TRUE)
    sliderInput("collab","Max Collaborations:",
               min=1, max=maxc, value=maxc, step=1)
  }
})
```

```

}
})

```

The above chunk shows the pubs/collabs slider. Based on the selected top view, it will change the slider to either show author publications or institution collaborations.

```

# --- render author view network ---
output$filteredGraph <- renderVisNetwork({
  req(input$view_mode=="Author view", input$selected_node)

  # extract ego-network for the selected author
  ego_id <- nodes$author_id[nodes$label==input$selected_node]
  ce <- subset(edges, from==ego_id|to==ego_id)
  ids <- unique(c(ce$from,ce$to,ego_id))
  cn <- subset(nodes, id %in% ids)

  pn <- subset(cn, id == ego_id)
  fn <- subset(cn, id != ego_id)

  fur_e <- subset(edges,
    (from %in% fn$id | to %in% fn$id) &
    !(from == ego_id | to == ego_id)
  )

  if(input$edge_mode == "Simple connections") {
    ae <- ce
  } else if (input$edge_mode == "Further connections") {
    ae <- (rbind(ce, fur_e))
  }

  fn <- fn %>% filter(institutions %in% input$inst_filter)
  fe <- subset(ae, from%in%fn$id & to%in%fn$id)

  if (input$net_mode=="Institution network") {
    # keep one central node + one per partner institution
    main_node <- pn

    fn <- fn %>% filter(pubs <= input$pubs_filter)
    fn <- fn %>% filter(continent %in% input$cont_filter)
    fn <- fn %>% filter(rank <= input$rank_filter | is.na(rank))
  }

```

```

inst_nodes <- fn[fn$id!=ego_id,] %>%
  distinct(institutions,.keep_all=TRUE) %>%
  mutate(
    id    = paste0("inst_",institutions),
    label = institutions,
    title = paste0("Institution: ",institutions)
  )
fn <- bind_rows(main_node,inst_nodes)
fe <- inst_nodes %>% transmute(from=ego_id,to=id,freq=1)
} else if (input$net_mode=="People network") {
  main_node <- pn

  fn <- fn %>% filter(pubs <= input$pubs_filter)
  fn <- fn %>% filter(continent %in% input$cont_filter)
  fn <- fn %>% filter(rank <= input$rank_filter | is.na(rank))

  fn <- bind_rows(main_node,fn)
  fe <- ae
}

if (nrow(fe)>0) {
  fe <- fe %>%
    mutate(
      width = scales::rescale(freq,to=c(1,10)),
      title = paste("Collaborations:",freq)
    )
}

visNetwork(fn,fe) %>%
  visLayout(randomSeed=1) %>%
  visEdges(smooth=TRUE) %>%
  visPhysics(
    solver="forceAtlas2Based",
    forceAtlas2Based=list(
      gravitationalConstant=-60,
      centralGravity=0.015,
      springLength=230,
      springConstant=0.08
    ),
    stabilization=FALSE
  ) %>%
  visEvents(stabilizationIterationsDone=

```

```

    "function(){ this.fit({animation:false}); }")
  })

```

And this chunk is the most important of the entire app. It is fully responsible for the creation of the graph given just the edges and nodes and all other user inputs like filters.

It first creates the `ego_id`, which is the ID of the selected node. Then using that, it finds all edges connected to the `ego_id` and by extension all nodes that are connected to the selected node. These nodes are the *connected nodes* or *cn* for short. Then it initializes the *primary node* (*pn*), and the *filtered nodes* (*fn*). Lastly, it creates the *further edges* (*fur_e*), based on the edges connecting the edges that connect all drawn nodes except for the *primary node*.

The first if-statement sees if the input is either only simple or further connections and uses either only *connected edges* or the bind of *connected edges* and *further edges*. Then it filters the *filtered nodes* by the institution filter and initializes *filtered edges*.

The second if-statement is tasked with dealing with the selection of either the people network or the institution network. If the institution view was chosen by the user, it will begin by filtering *filtered nodes* based on user input. Then it creates the `inst_nodes` by collapsing all institutions assigned to authors, keeping only distinct ones. After that, it binds them to the primary nodes. If the author view was chosen, it will filter the nodes the same way it did in the previous step. After the filter, it will bind the *primary node* to the *filtered nodes*.

The last if-statement checks if any edges are drawn and if yes, then it rescales them to have a more sensible width.

The last part is drawing the graph. It uses some parameters to make the graph look more stable and limit the physics to allow for better performance.

```

# --- render institution view map ---
output$inst_map <- renderLeaflet({
  req(input$view_mode=="Institution view", input$selected_inst)
  centre <- input$selected_inst

  link_tbl <- inst_edges %>%
    filter((inst_from==centre|inst_to==centre) & freq<=input$collab) %>%
    mutate(partner=ifelse(inst_from==centre,inst_to,inst_from))

  # on-the-fly geocode any new partners
  all_needed <- unique(c(centre,link_tbl$partner))
  to_geo <- setdiff(all_needed,
                    inst_coord %>% filter(!is.na(latitude)) %>% pull(address))
  if (length(to_geo)>0) {
    newc <- geocode(

```

```

    data.frame(address=to_geo,stringsAsFactors=FALSE),
    address,method="osm",lat="latitude",long="longitude"
  )
  inst_coord <- bind_rows(inst_coord,newc) %>%
    distinct(address,.keep_all=TRUE)
  saveRDS(inst_coord,cache_file)
}

pts      <- inst_coord %>%
  filter(address%in%all_needed & !is.na(latitude))
centre_pt <- pts %>% filter(address==centre)
partner_pts <- pts %>% filter(address!=centre)
vienna_pt <- partner_pts %>% filter(address==vienna_name)
other_pts <- partner_pts %>% filter(address!=vienna_name)

m <- leaflet() %>% addTiles()

# center marker
if (nrow(centre_pt)>0) {
  m <- m %>% addCircleMarkers(
    lng=centre_pt$longitude,lat=centre_pt$latitude,
    popup=centre,radius=10,color="red",fillOpacity=0.9
  )
}

# other partners
if (nrow(other_pts)>0) {
  m <- m %>% addCircleMarkers(
    lng=other_pts$longitude,lat=other_pts$latitude,
    popup=paste0(
      other_pts$address,"<br>Collaborations: ",
      link_tbl$freq[match(other_pts$address,link_tbl$partner)]
    ),
    radius=6,color="steelblue",fillOpacity=0.7
  )
}

# Vienna highlight
if (nrow(vienna_pt)>0) {
  m <- m %>% addCircleMarkers(
    lng=vienna_pt$longitude,lat=vienna_pt$latitude,
    popup=paste0(

```



```

vienna_name,"<br>Collaborations: ",
link_tbl$freq[match(vienna_name,link_tbl$partner)]
),
radius=8,color="orange",fillOpacity=0.9
)
}

# draw lines
if (nrow(centre_pt)>0 && nrow(partner_pts)>0) {
  for (i in seq_len(nrow(partner_pts))) {
    p <- partner_pts$address[i]
    col <- if (p==vienna_name) "orange" else "gray"
    w <- if (p==vienna_name) 3 else 1
    op <- if (p==vienna_name) 0.8 else 0.6
    m <- m %>% addPolylines(
      lng=c(centre_pt$longitude,partner_pts$longitude[i]),
      lat=c(centre_pt$latitude,partner_pts$latitude[i]),
      color=col,weight=w,opacity=op
    )
  }
}

# auto-zoom
if (nrow(centre_pt)>0 && nrow(partner_pts)>0) {
  all_lng <- c(centre_pt$longitude,partner_pts$longitude)
  all_lat <- c(centre_pt$latitude, partner_pts$latitude)
  m <- m %>% fitBounds(
    lng1=min(all_lng),lat1=min(all_lat),
    lng2=max(all_lng),lat2=max(all_lat)
  )
} else if (nrow(centre_pt)>0) {
  m <- m %>% setView(
    lng=centre_pt$longitude,lat=centre_pt$latitude,zoom=4
  )
} else if (nrow(partner_pts)>0) {
  m <- m %>% fitBounds(
    lng1=min(partner_pts$longitude),
    lat1=min(partner_pts$latitude),
    lng2=max(partner_pts$longitude),
    lat2=max(partner_pts$latitude)
  )
}
}

```

m
})